

Reviewer Pack — Secant Rank of Disjointness Communication Matrix Lower-Bound...

Entry c4fba4dde224 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 20:10:06 UTC

Secant Rank of Disjointness Communication Matrix Lower-Bounds Randomized Complexity

Entry ID: c4fba4dde224 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 20:10:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Secant Varieties of Veronese Embeddings)

Field B (complexity object): Randomized Communication Complexity of DISJOINTNESS

Statement:

For the DISJOINTNESS matrix $M_n \in \{0,1\}^{\{n \times n\}}$, its secant rank $\text{sr}(M_n)$ satisfies $\text{sr}(M_n) \geq \Omega(n)$. This follows from the fact that the Veronese variety $V_d \subseteq P^N$ ($d=2$) has secant variety $\sigma(V_d)$ of dimension $2d-1$, and M_n lies in $\sigma(V_d)$ only if $n \leq d^{\{1/2\}} + 1$.

Rationale (proposer's reasoning):

Secant rank captures the minimal number of rank-1 tensors needed to approximate a matrix, mirroring the communication complexity of DISJOINTNESS which requires $\Omega(n)$ bits. The Veronese embedding's secant variety dimension provides a geometric obstruction to low-rank approximations of the communication matrix.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2280403580fed75d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_disjointness_matrix(n):
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                M[i][j] = 1
                M[j][i] = 1
    return M

def transpose(A):
    return [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]

def matrix_multiply(A, B):
    C = [[0] * len(B[0]) for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(len(B[0])):
            for l in range(len(B)):
                C[i][j] += A[i][l] * B[l][j]
    return C

def rank(A):
    m, n = len(A), len(A[0])
    U = [row[:] for row in A]
    Vt = transpose([col[:] for col in A])
    k = min(m, n)
    for i in range(k):
        max_row = max(range(i, m), key=lambda r: abs(U[r][i]))
        if U[max_row][i] == 0:
            return i
        U[i], U[max_row] = U[max_row], U[i]
        Vt[i], Vt[max_row] = Vt[max_row], Vt[i]
        for j in range(i + 1, m):
            factor = U[j][i] / U[i][i]
            for l in range(n):
                U[j][l] -= factor * U[i][l]
                Vt[l][j] -= factor * Vt[l][i]
```

```

    return k

def secant_rank(M):
    n = len(M)
    rank_M = rank(M)
    if rank_M == n:
        return 1
    A = M[:rank_M][:rank_M]
    B = M[rank_M:][rank_M:]
    C = matrix_multiply(A, transpose(B))
    rank_C = rank(C)
    return rank_M + rank_C

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    M_n = generate_disjointness_matrix(n)
    sr_M_n = secant_rank(M_n)
    metric_value = sr_M_n
    instances_tested = 1
    conjecture_holds = sr_M_n >= 0.6 * n
    counterexample = "" if conjecture_holds else f"n={n}, sr(M_n)={sr_M_n}"
    return {
        "metric_name": "secant_rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 89)) # Default to first 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample_desc = results[0]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'secant_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'secant_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'secant_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'secant_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'secant_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_427ba9ca.py", line 90, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_427ba9ca.py", line 71, in run_trial
    sr_M_n = secant_rank(M_n)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_427ba9ca.py", line 63, in secant_rank
    C = matrix_multiply(A, transpose(B))
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_427ba9ca.py", line 28, in transpose
    return [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]
               ~~~^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during execution, preventing reliable evaluation of conjecture validity. | next: Implement error handling for matrix transposition and re-run tests with validated matrix dimensions

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111246
2	propose	ollama_remote	qwen3:8b	0	0	55218
3	propose	ollama_remote	qwen3:8b	0	0	110569
4	propose	ollama_remote	qwen3:8b	0	0	42583
5	preregistration	ollama_remote	qwen3:8b	0	0	24541
6	novelty	ollama_remote	qwen3:8b	0	0	28895
7	novelty	ollama_remote	qwen3:8b	0	0	12007
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16382
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10807
10	judge	ollama_remote	qwen3:8b	0	0	17502

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 429749 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/c4fba4dde224.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c4fba4dde224.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c4fba4dde224.tar.gz (if generated)